

Lab 0 - Outline

1. Colab Intro & Data Exploration Basics
2. Git & GitHub Crash Course
3. Weekly Task



These slides were created with AI assistance and fully reviewed and verified by the TAs.



2. Git & GitHub Crash Course

An Introduction to Version Control

What is Version Control?

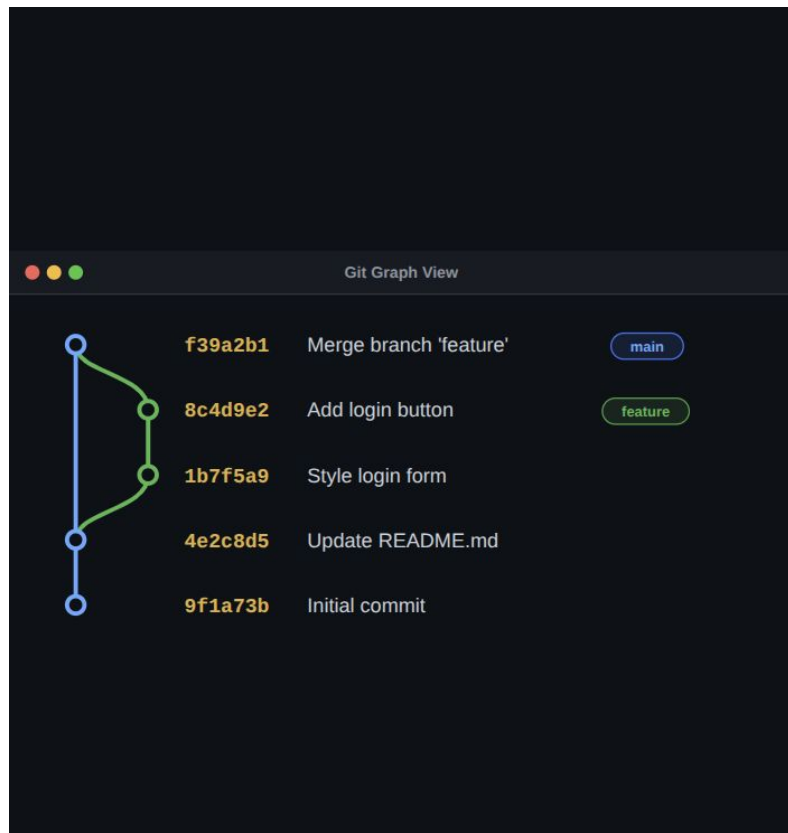
Have you ever saved files like this?



ml_project.ipynb
ml_project_FINAL.ipynb
ml_project_FINAL_v3_fixed.ipynb

Version Control Systems (VCS) solve this problem by:

- 1) **Tracking every change** made to your files over time.
- 2) Allowing you to seamlessly **revert to previous versions**.
- 3) Making **collaboration safe** and **organized**.



Git vs GitHub (Not the same!)



A **command-line program** installed on your local computer.

It **does the** actual **tracking** of changes to your code.

Think of Git as the camera taking the pictures.



A **website** that **hosts** your Git **repositories** in the cloud.

Makes it easy to **share**, **collaborate**, and **backup** your work.

Think of GitHub as Instagram where you share the pictures.

What is a terminal? (CLI vs. GUI)

The Command Line Interface (CLI)

A terminal is a **text-based interface**. Instead of clicking buttons, you **type instructions** (commands) directly to your computer and press Enter to execute them.

Why learn commands instead of using IDE buttons?

 **Learn the Principles:** You understand exactly what Git is doing under the hood, rather than relying on “magic” editor buttons.

 **Ultimate Adaptability:** Once you know the core CLI commands, you can instantly understand and reapply that knowledge to any Graphical User Interface (VS Code, IntelliJ, Visual Studio).



Step 1: Setup, Config & Authentication

1. Install Git & GitHub CLI

Download the tools from git-scm.com and cli.github.com (Windows users will also get **Git Bash** installed, a **Linux-style terminal** for handling Git commands).



2. Configure

```
# Set your name
git config --global user.name "Your Name"

# Set your email (must match your GitHub account!)
git config --global user.email "your.email@example.com"
```

3. Connect to GitHub

Run this command and follow the prompts to **log in securely via your browser**:

```
gh auth login
```

The Three States of Git

Git operates by moving files through three main "stages" locally:

1. **Working Directory:** Your actual files.
You edit them here.
2. **Staging Area:** The "loading dock". You might change 5 files, but only want to save 2 right now. You put those 2 into the Staging Area to group them together before saving.
3. **Repository (Commit):** The permanent, historical record of your saved changes.

Working Directory

Your computer's files (Editing)

`git add`



Staging Area

The "Loading Dock" (Index)

`git commit`



Local Repository

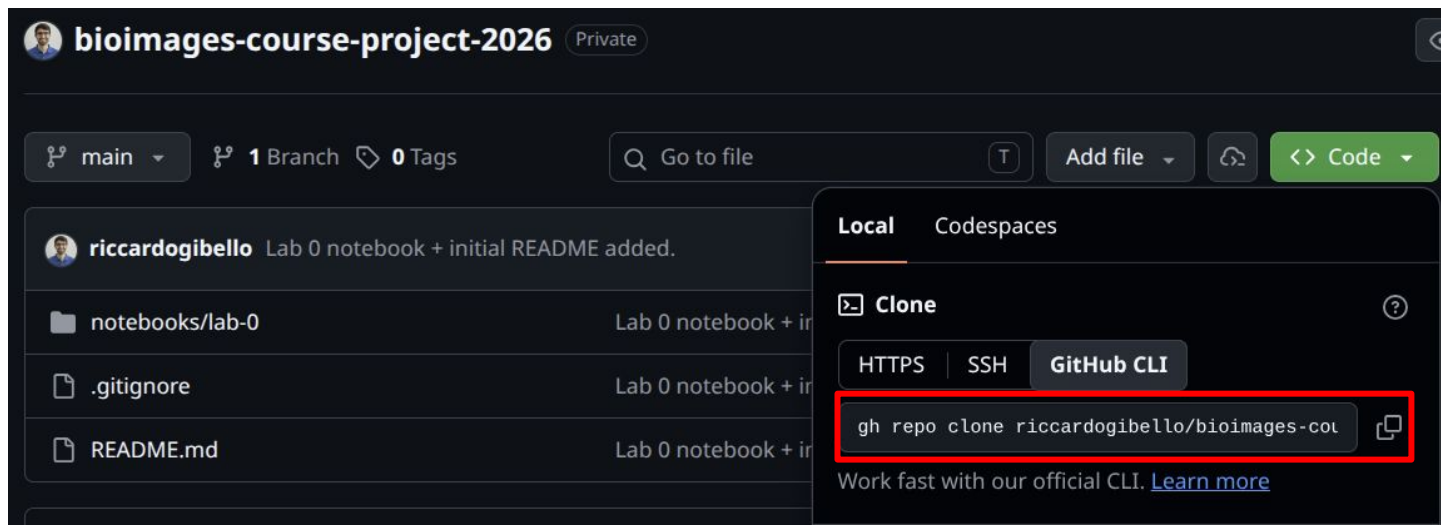
Permanent Timeline (.git folder)

Starting a Project

There are two primary ways to start working with Git:

Option A: Download an existing project

- Get a **copy of a repository from GitHub** to your computer:

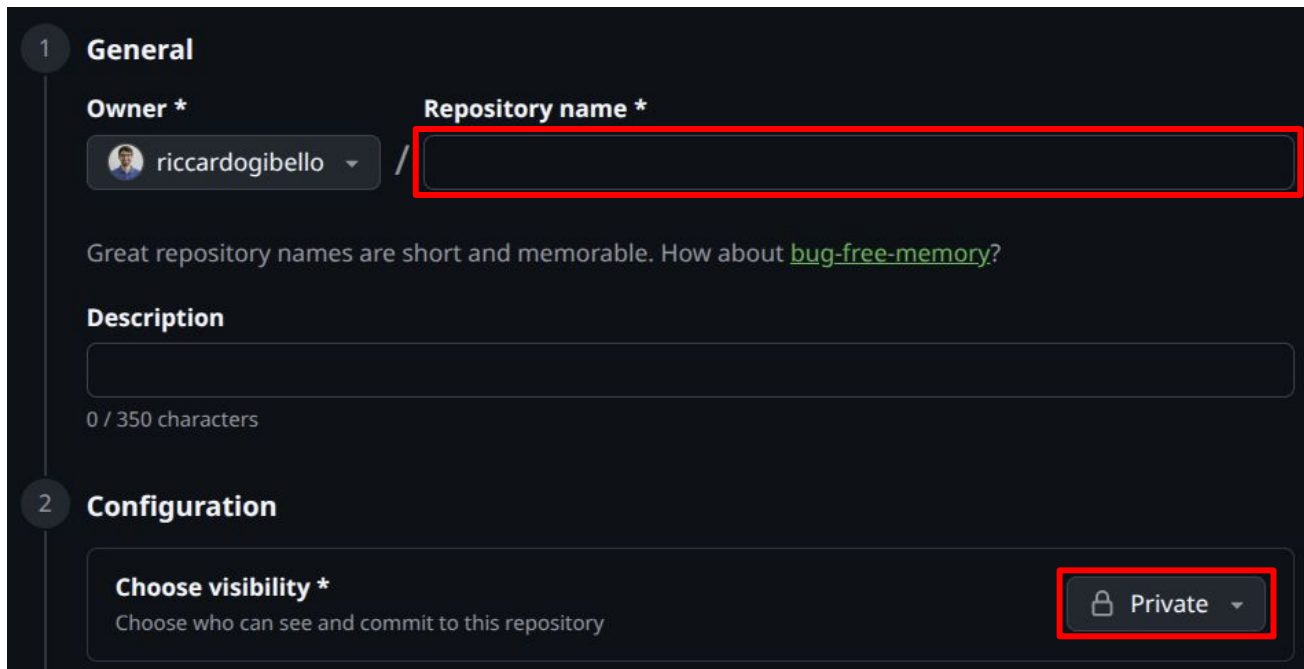


Starting a Project

There are two primary ways to start working with Git:

Option B: Start from scratch locally

- **Create a new remote GitHub repository** using their web interface:



The screenshot shows the GitHub repository creation interface. It is divided into two sections: '1 General' and '2 Configuration'. In the 'General' section, the 'Owner' is set to 'riccardogibello'. The 'Repository name' field is highlighted with a red rectangle. Below it, a suggestion for 'bug-free-memory?' is shown. The 'Description' field is empty, with a character count of '0 / 350 characters'. In the 'Configuration' section, the 'Choose visibility' dropdown is highlighted with a red rectangle, showing 'Private' as the selected option.

1 General

Owner * / **Repository name ***

riccardogibello

Great repository names are short and memorable. How about [bug-free-memory?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Private

Starting a Project

There are two primary ways to start working with Git:

Option B: Start from scratch locally

- **Create** in your **local** machine a new **folder** (possibly with the same name as of the repository to avoid confusion).
- **Follow** the terminal-based **instructions** on the screen to get your local folder and the remote one in sync.

```
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/riccardogibello/test.git
git push -u origin main
```



The Daily Workflow (Making Changes)

When you edit files, Git notices. Here is how you save them permanently:

1. Check Status

See what has changed:

```
git status
```

2. Stage Files

Add files to the Staging Area:

```
git add filename.txt # Specific file  
git add .             # ALL files
```

3. Commit Changes

Save the staged changes permanently with a descriptive message:

```
git commit -m "Added the login button to the homepage"
```

Syncing with GitHub

Once you have local commits, you need to share them or get updates from teammates.



Push (Upload)

Send your saved local commits up to GitHub so others can see them:

```
git push origin main
```

Pull (Download)

Download new commits from GitHub to your computer and merge them:

```
git pull origin main
```

Handling Merge Conflicts

What is a conflict?

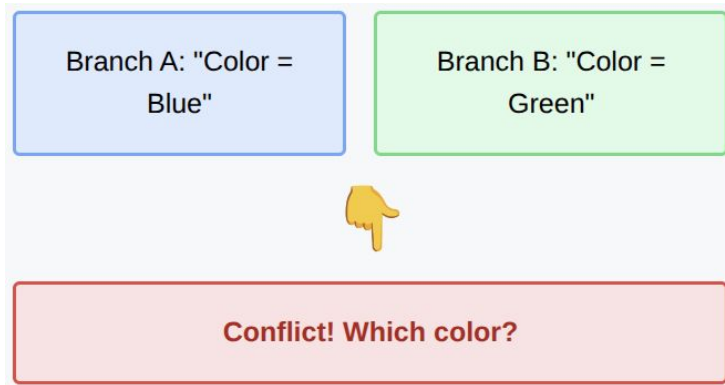
It happens when two people edit the exact same line in a file differently, and Git doesn't know whose version to keep.

When does it happen?

Usually when you run `git pull` or try to merge branches. Git will pause and output:

```
CONFLICT (content): Merge conflict in index.html
```

Don't panic! Git just needs your human brain to decide.



Example: Resolving a Conflict

Open the conflicted file in your editor (e.g., VS Code). Git added markers:

```
<h1>Welcome to our project!</h1>
<<<<<<< HEAD (Your current local changes)
<p>This text was written by Person A.</p>
=====
<p>This text was written by Person B.</p>
>>>>>>> new-feature-branch (The incoming changes)
```

How to fix it?

1. Decide which code to keep (or combine them manually).
2. **Delete the Git markers** (<<<<<<<, =====, >>>>>>>).
3. Save the file.
4. Run **git add index.html** to mark it resolved.
5. Run **git commit -m "Resolved merge conflict"** to finish.

Best Practices

 **Commit Often:** Commit every time you finish a small, logical chunk of work.

 **Write Good Commit Messages:** "Fixed the button color on homepage" is much better than "stuff" or "asdfghjkl".

 **Keep Organized:** Notebooks do not mix well with Git. Create one personal notebook per team member (e.g., *riccardo_sandbox.ipynb*). When a feature is ready, manually copy your working code into a single, shared *main_project.ipynb* file.

 **Pull Before You Push:** Always run **git pull** before you start working for the day to get your teammates' latest updates.

 **Don't Commit Secrets:** NEVER commit files containing passwords, API keys, or personal data. Use a **.gitignore** file!



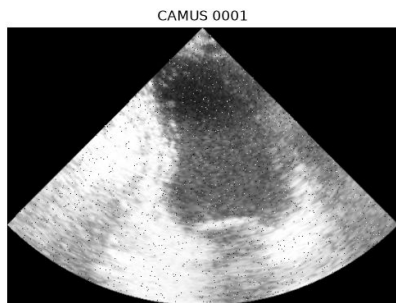
3. Hands-on Task on Image Preprocessing

Handling and Preprocessing Real-World DICOMs

Overview

- **Dataset:** 20 clinical images (CAMUS & Italian hospital)
- Includes **clean** and randomly **perturbed** samples.
- **Core Tasks** (useful project practice):
 - **Extract circular sector** (reusing/extending the provided code).
 - **Identify noise types** (following the theory lecture).
- **Do not upload** the data in **public GitHub repositories**.
- Segment masks and per-image noise will be shared for self-evaluation.
- **Google for Students** for Gemini assistance in Colab notebooks.

```
> camus
✓ inhouse
  ≡ 0011.dcm
  ≡ 0012.dcm
  ≡ 0013.dcm
  ≡ 0014.dcm
  ≡ 0015.dcm
  ≡ 0016.dcm
  ≡ 0017.dcm
  ≡ 0018.dcm
  ≡ 0019.dcm
  ≡ 0020.dcm
  ≡ answers.csv
```



```
=== CAMUS 0001 ===
Shape   : 386 x 545
Spacing : ['0.31', '0.31'] mm
dataset : camus
kind    : clean
view    : 2CH
phase   : ED
quality : Good
```

```
=== Inhouse 0011 ===
Shape   : 506 x 726
Spacing : ['0.31', '0.31'] mm
dataset : inhouse
kind     : clean
view     : 2CH
EDV_LV_ml : 227
EDV_LV_label : high/dilated
scar_label : 1
```

 [CLI Cheat Sheet & QA](#)

 [The Linux Command Line for
Beginners](#)

 [GitHub Education Git Cheat
Sheet \(PDF\)](#)

 [Interactive Game: Learn Git
Branching](#)

 [Pro Git Book \(Free Online\)](#)

Questions?